

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 12 — Unsupervised Learning

Exercises 4 and 11

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

1 Exercise 4: Single Linkage versus Complete Linkage

Hierarchical clustering starts with every observation in its own cluster and then repeatedly merges the two clusters that are closest according to a chosen linkage rule. The linkage rule does not change the pairwise distances between individual observations. It changes how we define the distance between two *clusters*.

For two clusters A and B , single linkage uses the closest pair of points:

$$D_{\text{single}}(A, B) = \min_{i \in A, j \in B} d(i, j).$$

Complete linkage uses the farthest pair of points:

$$D_{\text{complete}}(A, B) = \max_{i \in A, j \in B} d(i, j).$$

Because a maximum is always at least as large as a minimum,

$$D_{\text{complete}}(A, B) \geq D_{\text{single}}(A, B)$$

for every pair of clusters A and B . This simple inequality is the key to both parts of the exercise.

1.1 (a) The clusters $\{1, 2, 3\}$ and $\{4, 5\}$

The two methods merge the same two clusters at some point in their respective dendrograms. The question is whether the fusion occurs higher, at the same height, or whether there is not enough information to tell.

The answer is that the complete-linkage fusion occurs **higher than or at the same height** as the single-linkage fusion. It cannot occur lower.

The reason is direct. For these particular clusters, write

$$A = \{1, 2, 3\}, \quad B = \{4, 5\}.$$

Then

$$D_{\text{single}}(A, B) = \min\{d(1, 4), d(1, 5), d(2, 4), d(2, 5), d(3, 4), d(3, 5)\},$$

whereas

$$D_{\text{complete}}(A, B) = \max\{d(1, 4), d(1, 5), d(2, 4), d(2, 5), d(3, 4), d(3, 5)\}.$$

The maximum is at least the minimum. Since dendrogram height records the linkage distance at which a fusion occurs, the complete-linkage height is at least the single-linkage height.

It is important to distinguish this conclusion from the stronger statement that the heights must be different. They need not be. If all six cross-cluster distances happen to be equal, then their minimum and maximum are equal and both methods fuse the clusters at the same height. Without the actual pairwise distances, we cannot decide whether the inequality is strict.

Therefore, the precise answer is:

Complete linkage fuses $\{1, 2, 3\}$ and $\{4, 5\}$ at a height at least as large as single linkage.

1.2 (b) The clusters $\{5\}$ and $\{6\}$

Here each cluster contains only one observation:

$$A = \{5\}, \quad B = \{6\}.$$

There is only one cross-cluster distance, $d(5, 6)$. Therefore the minimum and maximum over the set containing this one value are identical:

$$D_{\text{single}}(\{5\}, \{6\}) = d(5, 6) = D_{\text{complete}}(\{5\}, \{6\}).$$

Consequently, the two methods fuse observations 5 and 6 at **exactly the same height**.

This conclusion does not depend on the locations of observations 5 and 6 or on the rest of the dataset. Whenever both clusters are singletons, every linkage definition based on the minimum or maximum of cross-cluster distances reduces to the same one pairwise distance.

Thus,

The single-linkage and complete-linkage fusions occur at the same height.

2 Exercise 11: Matrix Completion

Matrix completion estimates entries that are missing from a matrix by using low-dimensional structure in the entries that are observed. The central assumption is that the complete matrix is approximately low rank. In other words, although the matrix may have many rows and columns, most of its variation can be represented using a relatively small number of latent components.

Suppose X is the unknown complete matrix and Ω is the set of observed entries. We observe X_{ij} only for $(i, j) \in \Omega$. A rank- M approximation seeks a matrix $\hat{X}$ of the form

$$\hat{X}_M = U_M D_M V_M^T,$$

where U_M and V_M contain the first M singular vectors and D_M contains the first M singular values. Equivalently, the singular value decomposition gives

$$X \approx \sum_{m=1}^M d_m u_m v_m^T.$$

Each outer product $u_m v_m^T$ is a rank-one pattern. Increasing M allows more patterns to be represented, so approximation bias generally decreases; however, larger values of M can also fit noise and are more expensive to estimate when many entries are missing.

2.1 Algorithm 12.1 in plain language

The matrix-completion algorithm alternates between two operations:

1. fill the missing entries with current guesses;
2. compute a rank- M approximation of the completed matrix using the SVD, then use that approximation to update the missing entries.

The observed entries are never changed. They anchor the procedure to the actual data, while the unobserved entries are repeatedly replaced by values consistent with the estimated low-rank structure.

Let P_Ω be the projection that keeps observed entries and sets missing entries to zero. The update can be written conceptually as

$$X^{(t+1)} = P_\Omega(X) + P_{\Omega^c} \left(\mathcal{P}_M(X^{(t)}) \right),$$

where $\mathcal{P}_M$ means the best rank- M SVD approximation and Ω^c is the set of missing entries. The first term restores the observed values; the second supplies estimates only where values are missing.

The function below also tracks a convergence quantity. After each update, it computes the relative change between consecutive completed matrices:

$$\delta_t = \frac{\|X^{(t+1)} - X^{(t)}\|_F}{\max(\|X^{(t)}\|_F, \varepsilon)},$$

where $\|\cdot\|_F$ is the Frobenius norm and ε prevents division by zero. Iteration stops when δ_t is below `tol` or when `max_iter` is reached.

```
rank_m_approximation <- function(x, M) {
  decomposition <- svd(x, nu = M, nv = M)
  rank_used <- min(M, length(decomposition$d))

  decomposition$u[, seq_len(rank_used), drop = FALSE] %*%
    (decomposition$d[seq_len(rank_used)] *
     t(decomposition$v[, seq_len(rank_used), drop = FALSE]))
}

matrix_completion <- function(X_observed,
                              observed,
                              M,
```

```

        tol = 1e-5,
        max_iter = 1000,
        verbose = FALSE) {
stopifnot(is.matrix(X_observed), is.matrix(observed))
stopifnot(all(dim(X_observed) == dim(observed)))
stopifnot(length(M) == 1, M >= 1, M == as.integer(M))
stopifnot(length(tol) == 1, tol > 0)
stopifnot(length(max_iter) == 1, max_iter >= 1)

if (!all(is.finite(X_observed[observed]))) {
  stop("Observed entries must be finite.")
}
if (!any(observed)) {
  stop("At least one entry must be observed.")
}

# Column means provide a stable starting value for each missing entry.
completed <- X_observed
for (j in seq_len(ncol(completed))) {
  observed_in_column <- observed[, j]
  column_mean <- if (any(observed_in_column)) {
    mean(X_observed[observed_in_column, j])
  } else {
    mean(X_observed[observed], na.rm = TRUE)
  }
  completed[!observed_in_column, j] <- column_mean
}
completed[observed] <- X_observed[observed]

convergence <- numeric(max_iter)
approximation <- completed
iteration <- 0L

for (step in seq_len(max_iter)) {
  approximation <- rank_m_approximation(completed, M)
  updated <- completed
  updated[!observed] <- approximation[!observed]
  updated[observed] <- X_observed[observed]

  relative_change <- sqrt(sum((updated - completed)^2)) /
    max(sqrt(sum(completed^2)), .Machine$double.eps)
  convergence[step] <- relative_change
  iteration <- step

  if (verbose) {
    cat(sprintf("iteration %d: relative change = %.7g\n",
                step, relative_change))
  }
}

```

```

    }

    completed <- updated
    if (relative_change < tol) {
      break
    }
  }

  list(
    completed = completed,
    iterations = iteration,
    convergence = convergence[seq_len(iteration)],
    converged = convergence[iteration] < tol,
    rank = M
  )
}

```

The observed-entry constraint is worth checking explicitly. Even after many iterations, the returned matrix must reproduce every value that was supplied to the function:

```

set.seed(12)
toy <- matrix(rnorm(30), nrow = 6)
toy_observed <- matrix(runif(length(toy)) > 0.25,
                      nrow = nrow(toy), ncol = ncol(toy))

toy_input <- toy
toy_input[!toy_observed] <- NA_real_

toy_fit <- matrix_completion(toy_input, toy_observed, M = 2)

max_observed_difference <- max(abs(
  toy_fit$completed[toy_observed] - toy[toy_observed]
))
max_observed_difference

```

```
## [1] 0
```

The maximum difference should be zero up to floating-point precision. This check matters because a method that changes observed entries would no longer be completing the matrix; it would simply be fitting a low-rank approximation to a modified dataset.

2.2 Preparing the Boston experiment

The `Boston` dataset contains 506 observations and 13 numeric features. The exercise asks us to standardize the features first. For every feature j , the transformation is

$$Z_{ij} = \frac{X_{ij} - \bar{X}_j}{s_j},$$

so each column has mean approximately zero and standard deviation one. This is important because

the SVD is scale-sensitive: without standardization, a feature measured in large units could dominate the decomposition simply because of its units rather than because it carries more structure.

```
Boston_features <- as.matrix(ISLR2::Boston[, setdiff(names(ISLR2::Boston), "medv")])
Boston_scaled <- scale(Boston_features)

c(
  largest_absolute_column_mean = max(abs(colMeans(Boston_scaled))),
  largest_standard_deviation_error = max(abs(apply(Boston_scaled, 2, sd) - 1))
)

##      largest_absolute_column_mean largest_standard_deviation_error
##                        1.149882e-14                        8.215650e-15
```

We evaluate approximation error only on entries that were deliberately hidden. Because the original `Boston_scaled` matrix is available to us in this experiment, we can compare an imputed value with its known truth. For a mask H of hidden entries, the relative error is

$$\text{RelErr}(\hat{X}) = \frac{\|H \odot (\hat{X} - X)\|_F}{\|H \odot X\|_F},$$

where $\odot$ denotes elementwise multiplication. Evaluating on hidden entries is the matrix-completion analogue of using a test set: those values were not supplied to the algorithm.

2.3 One worked run

This first run hides 15% of the entries, uses a rank- $M = 3$ approximation, and prints the convergence progress. The hidden entries are created from a random permutation, so the missingness is random while still allowing us to construct nested masks for the full experiment below.

```
relative_error <- function(estimate, truth, hidden) {
  sqrt(sum((estimate[hidden] - truth[hidden])^2)) /
  max(sqrt(sum(truth[hidden]^2)), .Machine$double.eps)
}

set.seed(11)
missing_fraction <- 0.15
hidden_count <- ceiling(missing_fraction * length(Boston_scaled))
hidden_indices <- sample(seq_along(Boston_scaled), hidden_count)
hidden <- matrix(FALSE, nrow(Boston_scaled), ncol(Boston_scaled))
hidden[hidden_indices] <- TRUE
observed <- !hidden

input <- Boston_scaled
input[hidden] <- NA_real_
worked_run <- matrix_completion(input, observed, M = 3,
                                tol = 1e-5, max_iter = 500,
                                verbose = TRUE)

## iteration 1: relative change = 0.2397423
```

```
## iteration 2: relative change = 0.07734169
## iteration 3: relative change = 0.03435819
## iteration 4: relative change = 0.01913215
## iteration 5: relative change = 0.01216897
## iteration 6: relative change = 0.008320287
## iteration 7: relative change = 0.005921288
## iteration 8: relative change = 0.0043143
## iteration 9: relative change = 0.003190163
## iteration 10: relative change = 0.002382493
## iteration 11: relative change = 0.001792249
## iteration 12: relative change = 0.001356022
## iteration 13: relative change = 0.001031114
## iteration 14: relative change = 0.0007877646
## iteration 15: relative change = 0.0006047486
## iteration 16: relative change = 0.0004666875
## iteration 17: relative change = 0.0003623133
## iteration 18: relative change = 0.0002832998
## iteration 19: relative change = 0.0002234514
## iteration 20: relative change = 0.0001781273
## iteration 21: relative change = 0.0001438272
## iteration 22: relative change = 0.0001178911
## iteration 23: relative change = 9.828156e-05
## iteration 24: relative change = 8.342855e-05
## iteration 25: relative change = 7.212053e-05
## iteration 26: relative change = 6.342747e-05
## iteration 27: relative change = 5.664452e-05
## iteration 28: relative change = 5.124706e-05
## iteration 29: relative change = 4.685232e-05
## iteration 30: relative change = 4.318618e-05
## iteration 31: relative change = 4.005516e-05
## iteration 32: relative change = 3.732396e-05
## iteration 33: relative change = 3.489828e-05
## iteration 34: relative change = 3.271229e-05
## iteration 35: relative change = 3.071963e-05
## iteration 36: relative change = 2.888724e-05
## iteration 37: relative change = 2.719109e-05
## iteration 38: relative change = 2.561334e-05
## iteration 39: relative change = 2.41404e-05
## iteration 40: relative change = 2.27616e-05
## iteration 41: relative change = 2.146835e-05
## iteration 42: relative change = 2.025355e-05
## iteration 43: relative change = 1.911116e-05
## iteration 44: relative change = 1.803597e-05
## iteration 45: relative change = 1.702337e-05
## iteration 46: relative change = 1.606925e-05
## iteration 47: relative change = 1.516987e-05
## iteration 48: relative change = 1.432183e-05
## iteration 49: relative change = 1.3522e-05
```

```
## iteration 50: relative change = 1.27675e-05
## iteration 51: relative change = 1.205562e-05
## iteration 52: relative change = 1.138387e-05
## iteration 53: relative change = 1.074991e-05
## iteration 54: relative change = 1.015156e-05
## iteration 55: relative change = 9.586762e-06

list(
  iterations = worked_run$iterations,
  converged = worked_run$converged,
  final_relative_change = tail(worked_run$convergence, 1),
  hidden_entry_relative_error = relative_error(
    worked_run$completed, Boston_scaled, hidden
  )
)

## $iterations
## [1] 55
##
## $converged
## [1] TRUE
##
## $final_relative_change
## [1] 9.586762e-06
##
## $hidden_entry_relative_error
## [1] 0.7695113
```

The convergence value and the approximation error answer different questions. The convergence value asks whether successive imputations have stopped changing. The hidden-entry error asks whether the final imputations are close to the values that were actually hidden. Convergence is necessary for a stable iteration, but it does not by itself guarantee a small prediction error.

2.4 Nested missingness experiment

We now follow the exercise exactly. The missing fractions are 5%, 10%, ..., 30%. For each of 10 repetitions, one random ordering of all matrix entries is created. The first 5% of that ordering are hidden for the 5% experiment, the first 10% are hidden for the 10% experiment, and so on. Thus the missing sets are nested within each repetition.

For every missingness level and every $M \in \{1, 2, \dots, 8\}$, we run the completion algorithm and calculate the hidden-entry relative error.

```
set.seed(2026)
missing_fractions <- seq(0.05, 0.30, by = 0.05)
rank_values <- 1:8
repetitions <- 10

experiment_results <- vector("list", length(missing_fractions) *
  length(rank_values) * repetitions)
```

```

result_index <- 1L

for (replication in seq_len(repetitions)) {
  permutation <- sample(seq_along(Boston_scaled))

  for (missing_fraction in missing_fractions) {
    hidden_count <- ceiling(missing_fraction * length(Boston_scaled))
    hidden <- matrix(FALSE, nrow(Boston_scaled), ncol(Boston_scaled))
    hidden[permutation[seq_len(hidden_count)]] <- TRUE
    observed <- !hidden

    input <- Boston_scaled
    input[hidden] <- NA_real_

    for (M in rank_values) {
      fit <- matrix_completion(input, observed, M = M,
                              tol = 1e-5, max_iter = 500)

      experiment_results[[result_index]] <- data.frame(
        repetition = replication,
        missing_fraction = missing_fraction,
        M = M,
        approximation_error = relative_error(
          fit$completed, Boston_scaled, hidden
        ),
        iterations = fit$iterations,
        converged = fit$converged
      )
      result_index <- result_index + 1L
    }
  }
}

experiment_results <- do.call(rbind, experiment_results)
head(experiment_results)

```

```

##   repetition missing_fraction M approximation_error iterations converged
## 1           1           0.05 1           0.7433034           9      TRUE
## 2           1           0.05 2           0.7516467          13      TRUE
## 3           1           0.05 3           0.7270391          16      TRUE
## 4           1           0.05 4           1.6350666         500     FALSE
## 5           1           0.05 5           1.7487664         500     FALSE
## 6           1           0.05 6           1.7616430         500     FALSE

```

2.5 Average approximation error

```

error_summary <- aggregate(
  approximation_error ~ missing_fraction + M,
  data = experiment_results,
  FUN = mean
)
error_summary

```

```

##      missing_fraction M approximation_error
## 1             0.05 1             0.7662636
## 2             0.10 1             0.7687960
## 3             0.15 1             0.7737457
## 4             0.20 1             0.7779226
## 5             0.25 1             0.7822767
## 6             0.30 1             0.7828148
## 7             0.05 2             0.7581456
## 8             0.10 2             0.7637368
## 9             0.15 2             0.7799377
## 10            0.20 2             0.8033937
## 11            0.25 2             0.8165775
## 12            0.30 2             0.8343027
## 13            0.05 3             0.7379708
## 14            0.10 3             0.7524522
## 15            0.15 3             0.7854768
## 16            0.20 3             0.8336620
## 17            0.25 3             0.8749093
## 18            0.30 3             0.9949447
## 19            0.05 4             1.7498005
## 20            0.10 4             1.6564260
## 21            0.15 4             1.6482757
## 22            0.20 4             1.6551721
## 23            0.25 4             1.4300589
## 24            0.30 4             1.5397663
## 25            0.05 5             1.9430449
## 26            0.10 5             1.8863537
## 27            0.15 5             1.9310998
## 28            0.20 5             1.9883614
## 29            0.25 5             1.9311053
## 30            0.30 5             1.8999691
## 31            0.05 6             2.1594985
## 32            0.10 6             2.0696658
## 33            0.15 6             2.0094303
## 34            0.20 6             2.0588072
## 35            0.25 6             2.0335435
## 36            0.30 6             1.9541001
## 37            0.05 7             2.1272487
## 38            0.10 7             2.0804777
## 39            0.15 7             2.0266779

```

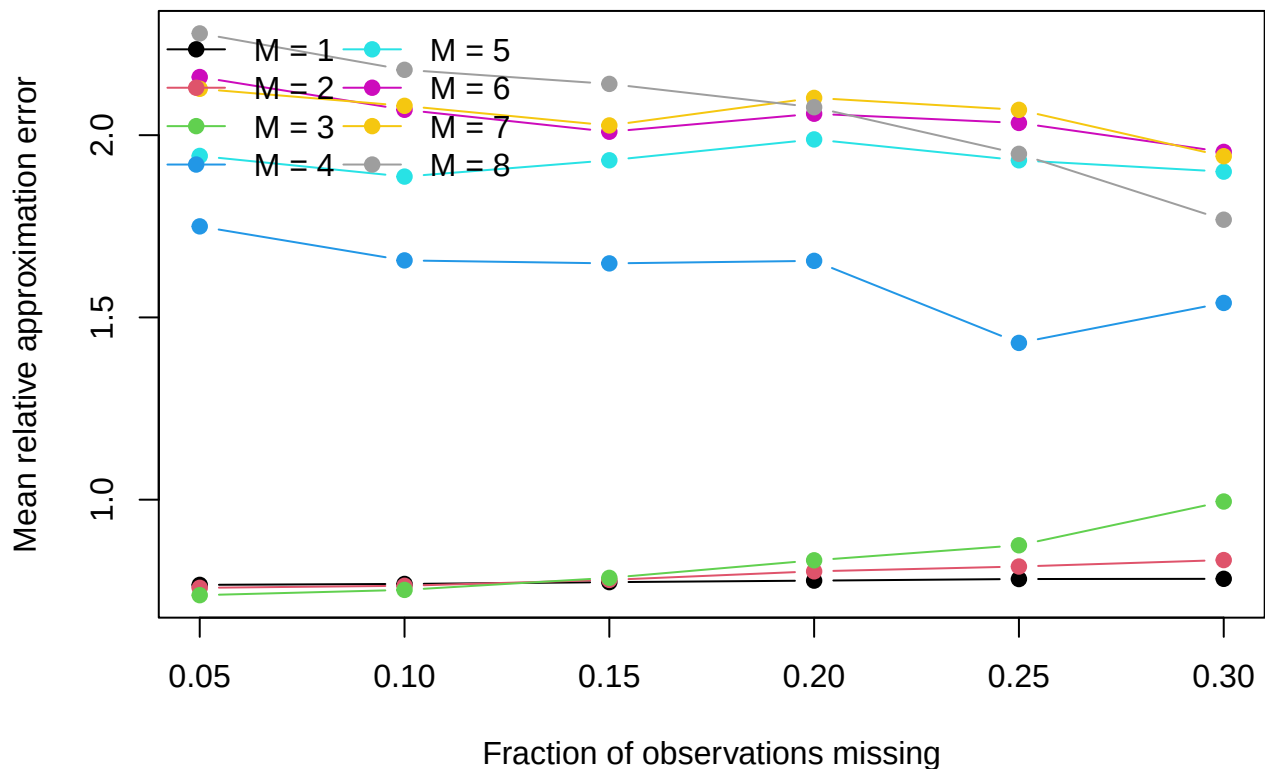
```
## 40          0.20 7          2.1022470
## 41          0.25 7          2.0694801
## 42          0.30 7          1.9422833
## 43          0.05 8          2.2791991
## 44          0.10 8          2.1793132
## 45          0.15 8          2.1406799
## 46          0.20 8          2.0766689
## 47          0.25 8          1.9487890
## 48          0.30 8          1.7681085
```

```
plot(NA, xlim = range(missing_fractions),
     ylim = range(error_summary$approximation_error),
     xlab = "Fraction of observations missing",
     ylab = "Mean relative approximation error",
     main = "Matrix-completion error across missingness and rank")

for (M in rank_values) {
  curve_data <- error_summary[error_summary$M == M, ]
  lines(curve_data$missing_fraction, curve_data$approximation_error,
        type = "b", pch = 19, col = M)
}

legend("topleft", legend = paste("M =", rank_values),
      col = rank_values, lty = 1, pch = 19, ncol = 2, bty = "n")
```

Matrix-completion error across missingness and rank



The expected pattern is that error increases as the missing fraction grows: with more entries hidden, the algorithm has less information from which to recover the latent structure. Increasing M often lowers error at first because the approximation is more flexible. However, the benefit can flatten or reverse when a high-rank approximation starts fitting idiosyncratic noise rather than the stable structure shared across the matrix.

The exact ranking of the curves is an empirical result, not a theorem. It depends on the random masks, the tolerance, and the structure of the data. The 10 repetitions reduce the chance that one unusually easy or difficult mask determines the conclusion.

2.6 Convergence diagnostics

The function was also required to track iteration count. We summarize the average number of iterations and the fraction of fits that reached the specified tolerance.

```
iteration_summary <- aggregate(
  iterations ~ missing_fraction + M,
  data = experiment_results,
  FUN = mean
)

convergence_summary <- aggregate(
  converged ~ missing_fraction + M,
  data = experiment_results,
  FUN = mean
)

list(
  mean_iterations = iteration_summary,
  fraction_converged = convergence_summary
)
```

```
## $mean_iterations
##   missing_fraction M iterations
## 1          0.05 1         8.8
## 2          0.10 1        11.4
## 3          0.15 1        14.9
## 4          0.20 1        21.2
## 5          0.25 1        25.6
## 6          0.30 1        40.9
## 7          0.05 2        13.1
## 8          0.10 2        20.9
## 9          0.15 2        34.3
## 10         0.20 2        68.5
## 11         0.25 2       98.2
## 12         0.30 2      229.1
## 13         0.05 3        20.8
## 14         0.10 3        31.0
## 15         0.15 3       90.6
```

## 16	0.20	3	215.0
## 17	0.25	3	347.0
## 18	0.30	3	455.3
## 19	0.05	4	442.3
## 20	0.10	4	477.2
## 21	0.15	4	490.9
## 22	0.20	4	500.0
## 23	0.25	4	500.0
## 24	0.30	4	500.0
## 25	0.05	5	500.0
## 26	0.10	5	500.0
## 27	0.15	5	500.0
## 28	0.20	5	500.0
## 29	0.25	5	500.0
## 30	0.30	5	500.0
## 31	0.05	6	500.0
## 32	0.10	6	500.0
## 33	0.15	6	500.0
## 34	0.20	6	500.0
## 35	0.25	6	500.0
## 36	0.30	6	500.0
## 37	0.05	7	500.0
## 38	0.10	7	500.0
## 39	0.15	7	500.0
## 40	0.20	7	500.0
## 41	0.25	7	500.0
## 42	0.30	7	500.0
## 43	0.05	8	500.0
## 44	0.10	8	500.0
## 45	0.15	8	500.0
## 46	0.20	8	500.0
## 47	0.25	8	500.0
## 48	0.30	8	500.0
##			
## \$fraction_converged			
## missing_fraction M converged			
## 1	0.05	1	1.0
## 2	0.10	1	1.0
## 3	0.15	1	1.0
## 4	0.20	1	1.0
## 5	0.25	1	1.0
## 6	0.30	1	1.0
## 7	0.05	2	1.0
## 8	0.10	2	1.0
## 9	0.15	2	1.0
## 10	0.20	2	1.0
## 11	0.25	2	1.0
## 12	0.30	2	0.9

## 13	0.05 3	1.0
## 14	0.10 3	1.0
## 15	0.15 3	1.0
## 16	0.20 3	0.8
## 17	0.25 3	0.6
## 18	0.30 3	0.2
## 19	0.05 4	0.2
## 20	0.10 4	0.2
## 21	0.15 4	0.1
## 22	0.20 4	0.0
## 23	0.25 4	0.0
## 24	0.30 4	0.0
## 25	0.05 5	0.0
## 26	0.10 5	0.0
## 27	0.15 5	0.0
## 28	0.20 5	0.0
## 29	0.25 5	0.0
## 30	0.30 5	0.0
## 31	0.05 6	0.0
## 32	0.10 6	0.0
## 33	0.15 6	0.0
## 34	0.20 6	0.0
## 35	0.25 6	0.0
## 36	0.30 6	0.0
## 37	0.05 7	0.0
## 38	0.10 7	0.0
## 39	0.15 7	0.0
## 40	0.20 7	0.0
## 41	0.25 7	0.0
## 42	0.30 7	0.0
## 43	0.05 8	0.0
## 44	0.10 8	0.0
## 45	0.15 8	0.0
## 46	0.20 8	0.0
## 47	0.25 8	0.0
## 48	0.30 8	0.0

A larger missing fraction can require more iterations because the observed entries provide a weaker anchor for the missing values. If a fit reaches `max_iter` before the tolerance is met, that is not automatically a failure: it is a diagnostic signal that the chosen tolerance or rank may require more iterations, or that the alternating SVD updates are converging slowly.

3 Conclusion

Exercise 4 illustrates how a precise definition of cluster distance controls the geometry of a dendrogram. Complete linkage cannot assign a lower distance to a pair of clusters than single linkage because a maximum cannot be below a minimum. For singleton clusters, both definitions reduce to the same pairwise distance.

Exercise 11 turns the low-rank idea into an algorithm. Missing entries are initialized, a rank- M SVD approximation is computed, and only the missing entries are replaced. The observed entries remain fixed throughout. The experiment then evaluates the method honestly on entries that were hidden from it, across nested missingness levels, ranks $M = 1$ through 8, and 10 random repetitions. This separates algorithmic convergence from predictive approximation quality and makes the effect of missingness visible.